

Data Warehousing The Complete Guide

Architecture, schema design, ETL/ELT, Kimball vs. Inmon, star & snowflake schemas, slowly changing dimensions, and how modern lakehouses relate to it all — explained from first principles to interview-ready depth.

- 01 What a Data Warehouse Is (and Isn't)
- 02 OLTP vs. OLAP
- 03 Data Warehouse Architecture & ETL/ELT
- 04 Kimball vs. Inmon Methodologies
- 05 Star Schema, Snowflake Schema & Facts/Dimensions
- 06 Slowly Changing Dimensions (SCD)
- 07 Data Warehouse vs. Data Lake vs. Lakehouse
- 08 Interview Preparation Cheat Sheet

What a Data Warehouse Is

A **data warehouse (DW)** is a centralized repository that stores integrated, historical data from multiple operational systems, structured specifically to support **analytics, reporting, and business decision-making** — not day-to-day transaction processing. It's the destination layer where raw, scattered operational data becomes clean, consistent, query-friendly information.

Bill Inmon, one of the founders of the field, defined a data warehouse as being:

Subject-Oriented

Integrated

Time-Variant

Non-Volatile

Breaking Down the Four Properties

Property	What It Means	Example
Subject-Oriented	Organized around business subjects (sales, customers, inventory) rather than around application processes.	A "Sales" subject area combines order, customer, and product data from many source systems.
Integrated	Data from different source systems is standardized into a consistent format, naming, and unit of measure.	"M"/"F" and "Male"/"Female" from two systems are unified into one coding standard.
Time-Variant	Data is stored with a time dimension, preserving history rather than only the latest state.	You can query "what was the customer's address in 2023?" not just today's address.
Non-Volatile	Once loaded, data is not updated or deleted in place — it's read-only for reporting and only appended to over time.	New snapshots or corrected records are added, not overwritten.

TIP

Think of a data warehouse as the company's long-term memory. Operational systems (like a point-of-sale app) only care about "now." A warehouse cares about "then, and now, and how things changed in between."

Why Not Just Query the Operational Database Directly?

This is one of the most common interview questions, and the answer has several angles:

- **Performance isolation** — analytical queries (scanning millions of rows, heavy aggregations) would slow down or lock the live transactional system.
- **Data model mismatch** — operational databases are normalized for fast, safe writes; analytics needs denormalized structures optimized for fast reads.
- **Fragmentation** — a business rarely has one operational system. Sales, support, marketing, and finance each have their own database; a warehouse is the single place they all converge.
- **History** — operational systems usually only store the current state. A warehouse preserves history for trend analysis.

Chapter Summary

- A data warehouse centralizes and integrates data from multiple systems for analytics.
- Inmon's four defining properties: subject-oriented, integrated, time-variant, non-volatile.
- Warehouses are separated from operational systems to protect performance and preserve history.

OLTP vs. OLAP

Every data system leans toward one of two workload types. Understanding this distinction is foundational — it explains almost every design decision in a data warehouse.

Aspect	OLTP (Operational)	OLAP (Analytical)
Full name	Online Transaction Processing	Online Analytical Processing
Purpose	Run the business day-to-day	Analyze the business over time
Typical operations	INSERT, UPDATE, DELETE — many small transactions	Complex SELECTs with joins, aggregations, window functions
Schema design	Highly normalized (3NF) to avoid redundancy	Denormalized (star/snowflake) for read speed
Data volume per query	A few rows	Millions of rows scanned/aggregated
Users	Front-line staff, applications, customers	Analysts, data scientists, executives
Examples	Banking app, e-commerce checkout, POS system	BI dashboards, sales trend reports, forecasting

INTERVIEW ANGLE

If asked "why do we denormalize a data warehouse when normalization is best practice in application databases?" — the answer is that the two systems optimize for opposite goals. OLTP optimizes for write integrity and avoiding duplication; OLAP optimizes for read speed, where a bit of redundancy is a fair trade for fewer joins.

A Concrete Example

Imagine an online store. The OLTP database has separate normalized tables: `orders`, `order_items`, `customers`, `products`, `addresses`. Placing one order touches five or six tables in milliseconds — perfect for OLTP.

Now imagine an analyst wants: "total revenue by product category, by month, for the last three years, broken down by customer region." In the OLTP schema this means joining six-plus normalized tables across millions of historical rows — slow, and risky to run against a live system. In a warehouse's star

schema, this becomes one fact table joined to two or three dimension tables, purpose-built for exactly this kind of question.

Data Warehouse Architecture & ETL/ELT

The Classic Layered Architecture



Layer	Role
Source Systems	OLTP databases, SaaS APIs, flat files, logs, IoT streams — the raw origin of data.
Staging Area	A temporary landing zone where raw data is copied as-is, before transformation. Useful for auditing, reprocessing, and decoupling extraction from transformation.
Data Warehouse (Core)	Cleaned, conformed, integrated data modeled as fact and dimension tables (often star schema).
Data Marts	Smaller, department-specific subsets of the warehouse (e.g. a "Finance Mart", a "Marketing Mart") tuned for a specific audience.

ETL vs. ELT

These are the two dominant patterns for moving data through the layers above.

	ETL (Extract → Transform → Load)	ELT (Extract → Load → Transform)
Where transformation happens	In a separate processing engine, before loading	Inside the warehouse itself, after loading raw data
Typical era / tools	Traditional (Informatica, SSIS, Talend)	Modern cloud-native (dbt + Snowflake/BigQuery/Databricks)
Raw data retained?	Often not — only transformed data is loaded	Yes — raw data lands first, transformations are recorded and repeatable
Scalability of transforms	Limited by ETL server compute	Scales with the warehouse's own elastic compute (MPP engines)
Fits medallion architecture?	Less naturally	Yes — bronze (raw) → silver (cleaned) → gold (modeled) is ELT in spirit

TIP

This maps directly onto medallion architecture: the **bronze layer** is essentially "load," the **silver layer** is early transformation/cleaning, and the **gold layer** — your fact and dimension tables — is where the final business-ready transform happens. ELT and medallion architecture are two views of the same idea.

A Simple ELT Load Example

sql

load_gold_fact_sales.sql

```
INSERT INTO gold.fact_sales (
    order_id, customer_key, product_key, order_date_key,
    quantity, unit_price, sales_amount
)
SELECT
    s.order_id,
    dc.customer_key,
    dp.product_key,
    dd.date_key,
    s.quantity,
    s.unit_price,
    s.quantity * s.unit_price AS sales_amount
FROM silver.sales s
JOIN gold.dim_customers dc ON s.customer_id = dc.customer_id
JOIN gold.dim_products dp ON s.product_id = dp.product_id
JOIN gold.dim_date dd ON s.order_date = dd.full_date;
```

Chapter Summary

- Data flows: Source → Staging → Warehouse → Marts.
- ETL transforms before loading; ELT loads raw data first and transforms inside the warehouse.
- Medallion architecture (bronze/silver/gold) is a modern implementation of the ELT pattern.

Kimball vs. Inmon

These are the two classic, competing philosophies for *how* to build a data warehouse. Nearly every DW textbook and interview touches on this distinction.

	Ralph Kimball (Bottom-Up)	Bill Inmon (Top-Down)
Starting point	Build data marts first, per business process	Build one central, normalized enterprise warehouse first
Schema style	Dimensional modeling — star schemas	Normalized (3NF) enterprise model, marts derived later
Speed to value	Fast — a single mart can ship in weeks	Slower — the enterprise model must be designed carefully upfront
Consistency risk	Marts can drift out of sync without governance ("conformed dimensions" needed)	Lower risk — single source of truth from the start
Best fit	Agile teams, evolving requirements, faster iteration	Large enterprises needing strict governance and a single integrated model

INTERVIEW ANGLE

A strong answer isn't "Kimball is better" — it's recognizing the trade-off: Kimball trades some enterprise-wide consistency for speed and simplicity; Inmon trades speed for rigor and a guaranteed single source of truth. Most modern cloud warehouses (using dbt, star schemas, and conformed dimensions) lean Kimball-style in practice, even at large scale.

Star Schema, Snowflake Schema & Fact/Dimension Tables

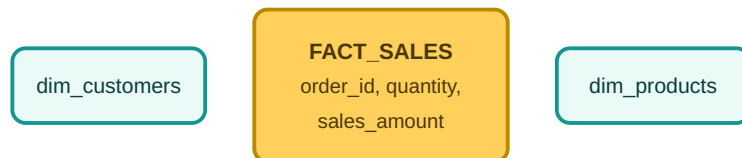
Fact Tables

A **fact table** stores the measurable, quantitative events of the business — the "verbs": a sale happened, a shipment happened, a click happened. It sits at the center of the schema and is typically the largest table by row count.

Dimension Tables

A **dimension table** stores the descriptive context around a fact — the "nouns": who, what, where, when. Dimensions answer "by what do we want to slice the facts?"

The Star Schema



(`dim_date` and `dim_geography` would sit above/below the fact table in the same pattern)

In a star schema, dimension tables are **denormalized** — each one is flat, with no further sub-tables. `dim_products` might repeat "category name" on every row instead of pointing to a separate categories table. This redundancy is intentional: it means a query only needs one join per dimension, which is fast.

The Snowflake Schema

A snowflake schema takes dimensions further and **normalizes** them into sub-dimensions — e.g. `dim_products` → `dim_category` → `dim_department`. This saves storage and reduces redundancy, but requires more joins per query.

Aspect	Star Schema	Snowflake Schema
Dimension structure	Denormalized (flat)	Normalized (split into sub-tables)
Query joins	Fewer, simpler	More, deeper
Query performance	Generally faster	Slightly slower due to extra joins
Storage footprint	Larger (redundant text)	Smaller (normalized)
Ease of understanding for BI users	Very easy — flat tables	Harder — more tables to navigate
Common in practice	Most common in modern cloud DWs	Used when storage was expensive / dimensions are huge

Types of Facts

Type	Behavior	Example
Additive SUM-able	Can be summed across any and all dimensions	sales_amount, quantity_sold
Semi-Additive Partial	Can be summed across some dimensions but not others (commonly not across time)	account_balance (sum across accounts, but not across days)
Non-Additive No SUM	Cannot be meaningfully summed at all	unit_price, profit_margin_percentage, ratios

Grain — The Most Important Design Decision

The **grain** of a fact table is the precise definition of what one row represents. Every design decision follows from getting this right first.

example

defining grain

```
-- Vague (avoid): "sales data"
-- Precise grain: "one row per product, per order line item"

fact_sales
  order_id      -- which order
  order_line_no -- which line within the order
  product_key   -- which product
  customer_key
  date_key
  quantity
  sales_amount
```

COMMON MISTAKE

Mixing grains in one fact table — e.g. some rows representing an order total and others representing a line item — silently breaks every SUM() a user writes later. Always fix the grain before writing a single line of the fact table's DDL.

Slowly Changing Dimensions (SCD)

Source systems change: a customer moves city, a product gets recategorized, an employee changes department. **Slowly Changing Dimensions** describe strategies for how a dimension table should react when its source data changes.

Type	Strategy	History Preserved?
SCD Type 0	Never update — the attribute is fixed forever once loaded (e.g. original signup date).	N/A — value never changes
SCD Type 1	Overwrite the old value with the new one. Simple, but history is lost.	No
SCD Type 2	Insert a new row for the changed record, with effective/expiry dates and a current-flag. The old row is kept and marked inactive.	Yes — full history
SCD Type 3	Add a new column to store the previous value alongside the current one (e.g. <code>current_city</code> , <code>previous_city</code>). Only one prior state is kept.	Partial — one prior value
SCD Type 4	Keep current values in the main dimension, move history into a separate history table.	Yes — split across two tables
SCD Type 6	Hybrid of Types 1, 2 and 3 — combine overwrite, new row, and previous-value column in one design.	Yes — most flexible

TIP

SCD Type 2 is by far the most commonly asked about in interviews and the most used in real warehouses, because it lets you answer "what did this look like at time X?" — the whole point of a time-variant warehouse.

SCD Type 2 in Practice

sql

dim_customers (SCD Type 2)

```
CREATE TABLE gold.dim_customers (  
  customer_key    INT      -- surrogate key,  
  customer_id     INT      -- natural/business key,  
  customer_name   VARCHAR(100),  
  city            VARCHAR(50),  
  effective_date  DATE,  
  expiry_date     DATE,  
  is_current      BOOLEAN  
);  
  
-- When a customer's city changes:  
-- 1) Expire the old row  
UPDATE gold.dim_customers  
SET expiry_date = CURRENT_DATE() - 1, is_current = FALSE  
WHERE customer_id = 501 AND is_current = TRUE;  
  
-- 2) Insert the new row  
INSERT INTO gold.dim_customers  
VALUES (9981, 501, 'Sara Ahmed', 'Giza', CURRENT_DATE(), NULL, TRUE);
```

Notice the **surrogate key** (`customer_key`) is different from the **natural key** (`customer_id`). This is essential: fact tables join on the surrogate key, so a fact row from before the change automatically joins to the old ("Cairo") version, and a fact row from after joins to the new ("Giza") version — history is preserved correctly.

Data Warehouse vs. Data Lake vs. Lakehouse

Aspect	Data Warehouse	Data Lake	Lakehouse
Data structure	Structured only	Structured, semi-structured, unstructured	All types, with structure enforced at the gold layer
Schema	Schema-on-write	Schema-on-read	Both, depending on layer (bronze/silver/gold)
Storage cost	Higher (proprietary formats)	Low (cheap object storage: S3, ADLS, HDFS)	Low, same as data lake
Primary users	BI analysts	Data scientists, ML engineers	Both
ACID transactions	Yes	No (traditionally)	Yes, via Delta Lake / Iceberg / Hudi
Typical engine	Snowflake, Redshift, BigQuery	HDFS + Spark, S3 + Spark	Databricks (Delta Lake), Snowflake (Iceberg)

TIP

A **lakehouse** is best understood as "a data lake that learned the warehouse's best habits" — cheap, flexible storage from the lake world, plus the ACID guarantees, schema enforcement, and fast BI-style querying of the warehouse world. Medallion architecture (bronze/silver/gold) is the standard way of organizing a lakehouse.

Interview Preparation Cheat Sheet

Q: WHAT'S THE DIFFERENCE BETWEEN A DATA WAREHOUSE AND A DATABASE?

A database (especially OLTP) is optimized for transactional writes and current-state storage; a data warehouse is optimized for analytical reads across large volumes of historical, integrated data.

Q: WHY USE SURROGATE KEYS INSTEAD OF NATURAL KEYS IN DIMENSIONS?

Surrogate keys are stable, warehouse-generated integers that don't depend on the source system, support SCD Type 2 history, join faster than composite/string keys, and insulate the warehouse from source-system key changes or reuse.

Q: WHAT IS A CONFORMED DIMENSION?

A dimension (like `dim_date` or `dim_customer`) built once and reused consistently across multiple fact tables or data marts, so that results from different reports stay consistent and comparable.

Q: WHAT IS A FACTLESS FACT TABLE?

A fact table with no measures — it just records that an event occurred, using only foreign keys. Example: a table recording which students attended which classes, used to answer "who did/didn't attend" questions.

Q: HOW WOULD YOU HANDLE LATE-ARRIVING DIMENSION RECORDS?

Insert a placeholder/"unknown member" row in the dimension so the fact load doesn't fail on a missing foreign key, then backfill the dimension attributes once the real record arrives — often paired with an SCD Type 2 update.

Whole-Guide Summary

- A data warehouse is a subject-oriented, integrated, time-variant, non-volatile store built for analytics.
- OLTP handles the business in real time; OLAP analyzes it in aggregate.
- Data flows Source → Staging → Warehouse → Marts, via ETL or (increasingly) ELT — the same pattern medallion architecture implements as bronze/silver/gold.
- Kimball (bottom-up, dimensional) and Inmon (top-down, normalized) are the two classic design philosophies.
- Star schemas denormalize dimensions for fast, simple joins; snowflake schemas normalize them to save space.
- Grain must be fixed before designing a fact table; facts are additive, semi-additive, or non-additive.
- SCD Type 2 is the standard technique for preserving dimension history using surrogate keys.
- Lakehouses combine data-lake flexibility with warehouse-grade reliability and performance.